

ASSIGNMENT

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: Assignment

Student Name: M. Niharika

Roll No.: A24126552097

1. TITLE

Implementation of Types of Inheritance in JavaScript

2. AIM

To implement the five types of inheritance — **single, multilevel, hierarchical, multiple, and hybrid** — in JavaScript using classes, the extends keyword, and mixins, with a real-time example for each.

3. OBJECTIVE

The objectives of this assignment are:

- To understand inheritance as a way of reusing code between related classes.
 - To use the extends keyword to create a child class from a parent class.
 - To implement single inheritance (Employee → Manager).
 - To implement multilevel inheritance (Person → Employee → Developer).
 - To implement hierarchical inheritance (Vehicle → Car and Bike).
 - To simulate multiple inheritance using mixins and Object.assign(), since a JavaScript class can extend only one parent.
 - To combine class inheritance with mixins to build hybrid inheritance (Employee → Developer / Tester, plus RemoteWork).
 - To verify by running the program that each object can call both its own and its inherited methods.
-

4. THEORY

Inheritance lets a child class reuse the properties and methods of a parent class instead of rewriting them. In JavaScript this is done with the *extends* keyword; behind the scenes the child's prototype is linked to the parent's prototype, so a method not found on the child is looked up on the parent.

The five types implemented in this assignment are:

- Single inheritance: one child class extends one parent class (Employee → Manager).
- Multilevel inheritance: a chain of classes, where a child becomes the parent of another class (Person → Employee2 → Developer). Developer can use methods from both classes above it.
- Hierarchical inheritance: several child classes extend the same parent (Vehicle → Car, Vehicle → Bike). Both children share the parent's start() method.
- Multiple inheritance: one class gets behaviour from more than one source. JavaScript classes can extend only one parent, so this is simulated with mixins — plain objects of methods copied onto a class's prototype using Object.assign().
- Hybrid inheritance: a combination of two or more types. Here Developer2 and Tester both extend Employee3 (hierarchical), and Developer2 also receives the RemoteWork mixin (multiple).

Object.assign(target, ...sources) copies the methods of each source object onto the target. Applying it to Class.prototype makes those methods available to every instance of the class, exactly as if they had been written inside it.

5. ALGORITHM / PROCEDURE

- Create a file inheritance.js and print a heading for each inheritance type using console.log().
- Single: define Employee with work(), define Manager extends Employee with manageTeam(); create a Manager and call both methods.
- Multilevel: define Person with introduce(), Employee2 extends Person with work(), Developer extends Employee2 with writeCode(); call all three methods on a Developer.
- Hierarchical: define Vehicle with start(), then Car and Bike that both extend Vehicle with drive() and ride(); call the inherited and own methods on each.
- Multiple: define the Camera and MusicPlayer mixin objects and a Smartphone class with makeCall().
- Copy both mixins onto Smartphone.prototype using Object.assign(); create a Smartphone and call makeCall(), takePhoto(), and playMusic().
- Hybrid: define Employee3 with login(), and Developer2 and Tester that extend it; define the RemoteWork mixin and copy it onto Developer2.prototype.
- Create a Developer2 and a Tester and call login() plus each object's own methods, and workFromHome() on the developer.

- Run the program with node inheritance.js and compare the output with the expected results.

6. PROGRAM

inheritance.js

```
// 1. SINGLE INHERITANCE
// Real-time Example: Employee -> Manager

console.log("1. SINGLE INHERITANCE");

class Employee {
  work() {
    console.log("Employee is working.");
  }
}

class Manager extends Employee {
  manageTeam() {
    console.log("Manager is managing the team.");
  }
}

let manager = new Manager();

manager.work();
manager.manageTeam();

// 2. MULTILEVEL INHERITANCE
// Real-time Example: Person -> Employee -> Developer

console.log("\n2. MULTILEVEL INHERITANCE");

class Person {
  introduce() {
    console.log("I am a person.");
  }
}

class Employee2 extends Person {
  work() {
    console.log("Employee is working.");
  }
}

class Developer extends Employee2 {
  writeCode() {
    console.log("Developer is writing code.");
  }
}

let developer = new Developer();
```

```
developer.introduce();
developer.work();
developer.writeCode();

// 3. HIERARCHICAL INHERITANCE
// Real-time Example: Vehicle -> Car / Bike

console.log("\n3. HIERARCHICAL INHERITANCE");

class Vehicle {
  start() {
    console.log("Vehicle is starting.");
  }
}

class Car extends Vehicle {
  drive() {
    console.log("Car is driving.");
  }
}

class Bike extends Vehicle {
  ride() {
    console.log("Bike is being ridden.");
  }
}

let car = new Car();
let bike = new Bike();

car.start();
car.drive();

bike.start();
bike.ride();

// 4. MULTIPLE INHERITANCE
// Using Mixins
// Real-time Example: SmartPhone

console.log("\n4. MULTIPLE INHERITANCE");

const Camera = {
  takePhoto() {
    console.log("Smartphone is taking a photo.");
  }
};

const MusicPlayer = {
  playMusic() {
    console.log("Smartphone is playing music.");
  }
};

class Smartphone {
  makeCall() {
```

```

        console.log("Smartphone is making a call.");
    }
}

// Adding features from both Camera and MusicPlayer
Object.assign(Smartphone.prototype, Camera, MusicPlayer);

let phone = new Smartphone();

phone.makeCall();
phone.takePhoto();
phone.playMusic();

// 5. HYBRID INHERITANCE
// Combination of inheritance + mixins
// Real-time Example: Company Employees

console.log("\n5. HYBRID INHERITANCE");

class Employee3 {
    login() {
        console.log("Employee logged into the company system.");
    }
}

class Developer2 extends Employee3 {
    writeCode() {
        console.log("Developer is writing code.");
    }
}

class Tester extends Employee3 {
    testSoftware() {
        console.log("Tester is testing the software.");
    }
}

const RemoteWork = {
    workFromHome() {
        console.log("Employee is working from home.");
    }
};

// Developer gets RemoteWork feature
Object.assign(Developer2.prototype, RemoteWork);

let developer2 = new Developer2();
let tester = new Tester();

developer2.login();
developer2.writeCode();
developer2.workFromHome();

tester.login();
tester.testSoftware();

```

7. CODE EXPLANATION

Code	Explanation
class Manager extends Employee	Creates Manager as a child of Employee; Manager objects can call work() as well as manageTeam().
class Developer extends Employee2	Extends a class that itself extends Person, forming a chain (multilevel inheritance).
class Car extends Vehicle / class Bike extends Vehicle	Two children share one parent, so both inherit start() (hierarchical inheritance).
const Camera = { takePhoto() {...} }	A mixin: a plain object holding methods that can be added to any class.
Object.assign(Smartphone.prototype, Camera, MediaPlayer)	Copies the methods of both mixins onto Smartphone's prototype, giving every Smartphone both abilities (simulated multiple inheritance).
class Developer2 extends Employee3	Inherits login() from Employee3 (the class-based part of hybrid inheritance).
Object.assign(Developer2.prototype, RemoteWork)	Adds workFromHome() to Developer2 only; Tester does not get it (the mixin part of hybrid inheritance).
new Manager() / new Car() / ...	Creates an object of the class so its own and inherited methods can be called.
console.log("\n2. ...")	Prints a heading; the \n adds a blank line to separate the outputs of each section.

8. TEST CASES AND EXECUTION DATA

The program was run with Node.js and the console output for each inheritance type was checked against the expected result.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Call work() and manageTeam() on a Manager	Inherited work() and own manageTeam() both run	Both messages printed	Pass
TC-02	Call introduce(), work(), writeCode() on a Developer	Methods from all three levels of the chain run	All three messages printed	Pass
TC-03	Call start() on a Car and on a Bike	Both children run the shared parent method start()	"Vehicle is starting." printed for both	Pass
TC-04	Call makeCall(), takePhoto(), playMusic() on a Smartphone	Own method plus both mixin methods run	All three messages printed	Pass
TC-05	Call login(), writeCode(), workFromHome() on a Developer2	Inherited, own, and mixin methods all run	All three messages printed	Pass

Test Case	Action	Expected Result	Actual Result	Status
TC-06	Call login() and testSoftware() on a Tester	Inherited and own methods run	Both messages printed	Pass

9. OUTPUT

Output 1: Single Inheritance

```
Terminal - node

$ node inheritance.js
1. SINGLE INHERITANCE
Employee is working.
Manager is managing the team.
$
```

Output 2: Multilevel Inheritance

```
Terminal - node

$ node inheritance.js
2. MULTILEVEL INHERITANCE
I am a person.
Employee is working.
Developer is writing code.
$
```

Output 3: Hierarchical Inheritance

```
Terminal - node

$ node inheritance.js
3. HIERARCHICAL INHERITANCE
Vehicle is starting.
Car is driving.
Vehicle is starting.
Bike is being ridden.
$
```

Output 4: Multiple Inheritance (Mixins)

```
Terminal - node
$ node inheritance.js
4. MULTIPLE INHERITANCE
Smartphone is making a call.
Smartphone is taking a photo.
Smartphone is playing music.
$
```

Output 5: Hybrid Inheritance

```
Terminal - node
$ node inheritance.js
5. HYBRID INHERITANCE
Employee logged into the company system.
Developer is writing code.
Employee is working from home.
Employee logged into the company system.
Tester is testing the software.
$
```

10. RESULT

Thus, single, multilevel, hierarchical, multiple, and hybrid inheritance were implemented in JavaScript and verified by running the program. Every object could call its own methods as well as the methods it inherited, and all test cases passed.

Observation: JavaScript classes support only one parent through extends, so true multiple inheritance is not available; mixins with `Object.assign()` achieve the same effect by copying methods onto the prototype. Note that the copied methods are independent of any class hierarchy, so `instanceof` will not report the mixin as a parent of the class.